



Do conceito ao código:

Design de bancos de dados com pgModeler 2.0

Raphael Araújo e Silva <raphael@pgmodeler.io>

Raphael Araújo e Silva

- 🔑 Criador do **pgModeler** e fundador da **Nullptr Labs**, empresa que assumiu em 2026 o desenvolvimento da ferramenta;
- 🎓 Bacharel em **Ciência da Computação** pela Universidade Federal do Tocantins. Pós-graduado em Rede de Computadores e Software Livre pela Universidade Católica do Tocantins;
- 🏛️ Há 20 anos como Analista e Desenvolvedor de Software na **Assembleia Legislativa do Estado do Tocantins**;
- ♥️ Fascinado pela programação, teoria de compiladores, computação gráfica, processamento de imagens e entusiasta da área de banco de dados.



Motivações para o projeto

Modelagem e Documentação

Ferramentas genéricas cross-database não suficientemente focadas em modelagem de banco de dados considerando todas as nuances do PostgreSQL.

Custo x Benefício

Softwares de modelagem proprietários absurdamente caros somado a isso, havia a escassez de ferramentas free ou open-source robustas.

Especificidades do PostgreSQL

Ausência de recursos avançados do PostgreSQL. Features como funções, triggers, herança, extensões, tipos de dados customizados não eram suportados.

Geração de código SQL compatível

Muitas vezes, códigos SQL gerados por ferramentas de modelagem em uso precisavam de intervenções manuais para serem compatíveis com PostgreSQL.

cout << "Hello World!";

★ Em 2006 nascia o projeto PostgreSQL Database Modeler ou pgsqlDBM. Anos depois alterado para pgModeler;

📄 Concebido como um projeto pessoal sem intenção de divulgação, porém, em **2012** o código-fonte foi liberado sob licença **GPL2** e posteriormente atualizado para **GPL3**;

≡ Multiplataforma, implementado em **ISO C++ 17** e framework **Qt** **6.10**. Oficialmente suportado nos três principais SOs: **Linux**, **Windows** e **mac OS**.



0.1.0



0.1.1 - 0.9.0



pgModeler
0.5.1 - 0.9.4



pgModeler
1.0.0+



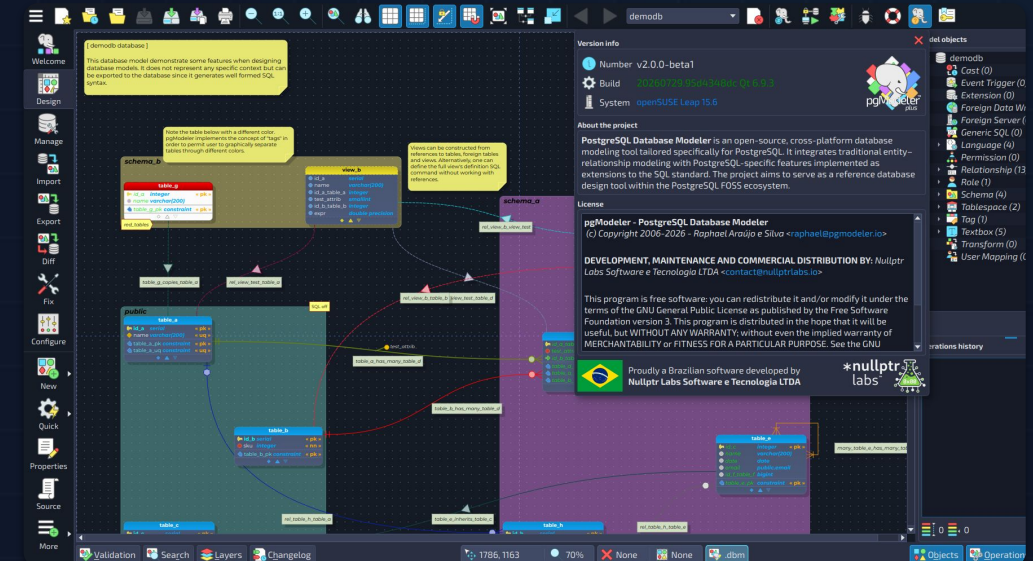
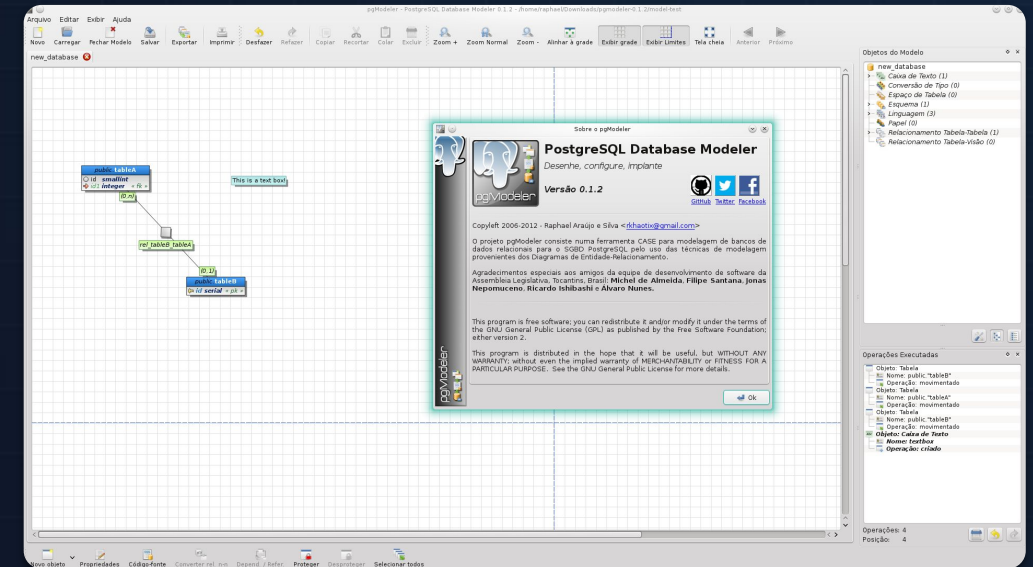
pgModeler plus
1.2.0+ (paid ver.)



pgModeler community
2.0.0+



pgModeler plus
2.0.0+ (paid ver.)



Hands on!

Instalação e tour pela ferramenta

Download e instalação do pgModeler Plus, além de uma breve apresentação das principais funcionalidades da ferramenta as quais serão abordadas neste workshop.

Modelagem do banco de dados “rentacar”

Criação de um modelo para controle de aluguel de veículos e apresentação recursos como visualização de código, propagação de colunas, validação dentre outros.

Exportação e engenharia reversa

Apresentação dos modos de exportação e em seguida faremos uso da engenharia reversa para reconstruir o modelo “rentacar” a partir do banco de dados “vivo”.

Comparação de banco de dados (diff)

Demonstração de como o pgModeler pode ser usado para criar scripts SQL para sincronização do banco de dados “vivo” a partir do modelo original.

Hands on!

Administração de banco de dados

Apresentação e uso do módulo de administração de banco de dados e como ele pode ser usado para manipular dados e objetos de banco de dados.

Ferramenta CLI

Introdução ao pgmodeler-cli, ferramenta de linha de comando que disponibiliza operações da GUI em ambiente terminal.

Visão geral dos plugins

Uma rápida passagem pelos plugins disponíveis na versão comercial do pgModeler e como eles o tornam ainda mais poderoso e produtivo.

Perguntas e respostas

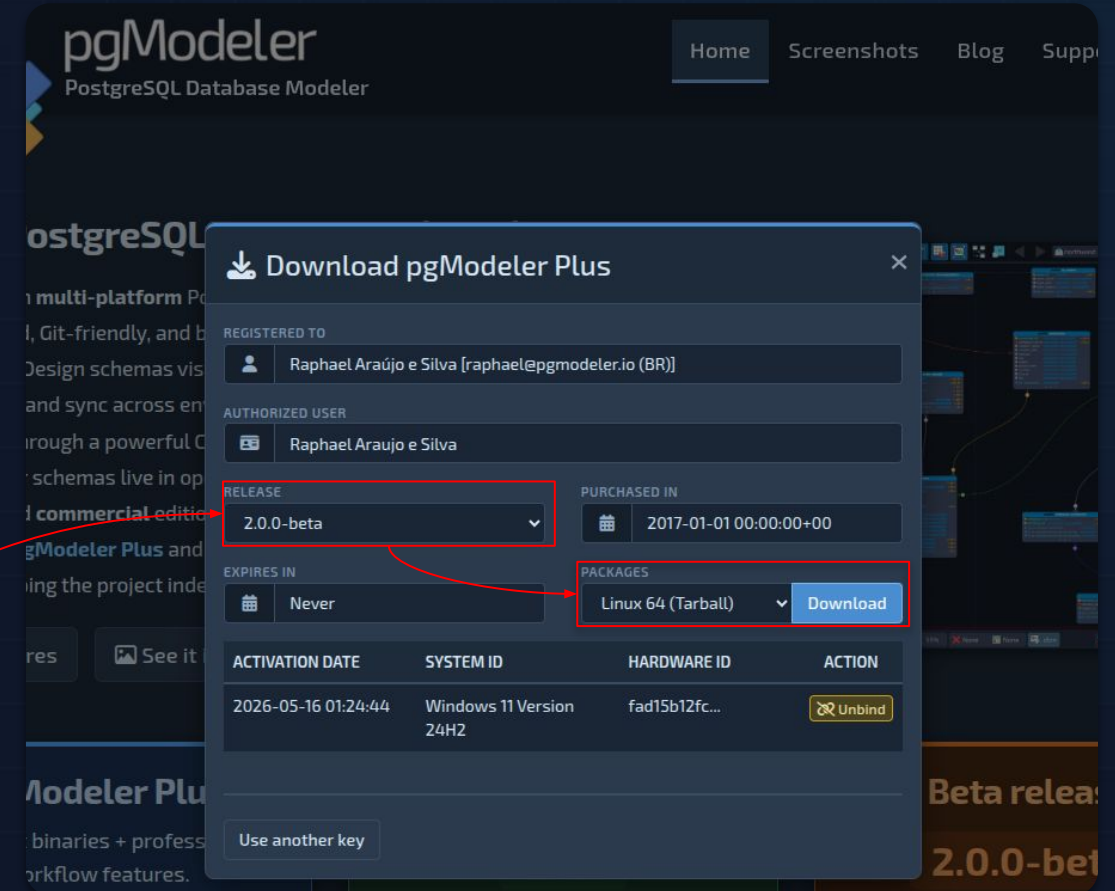
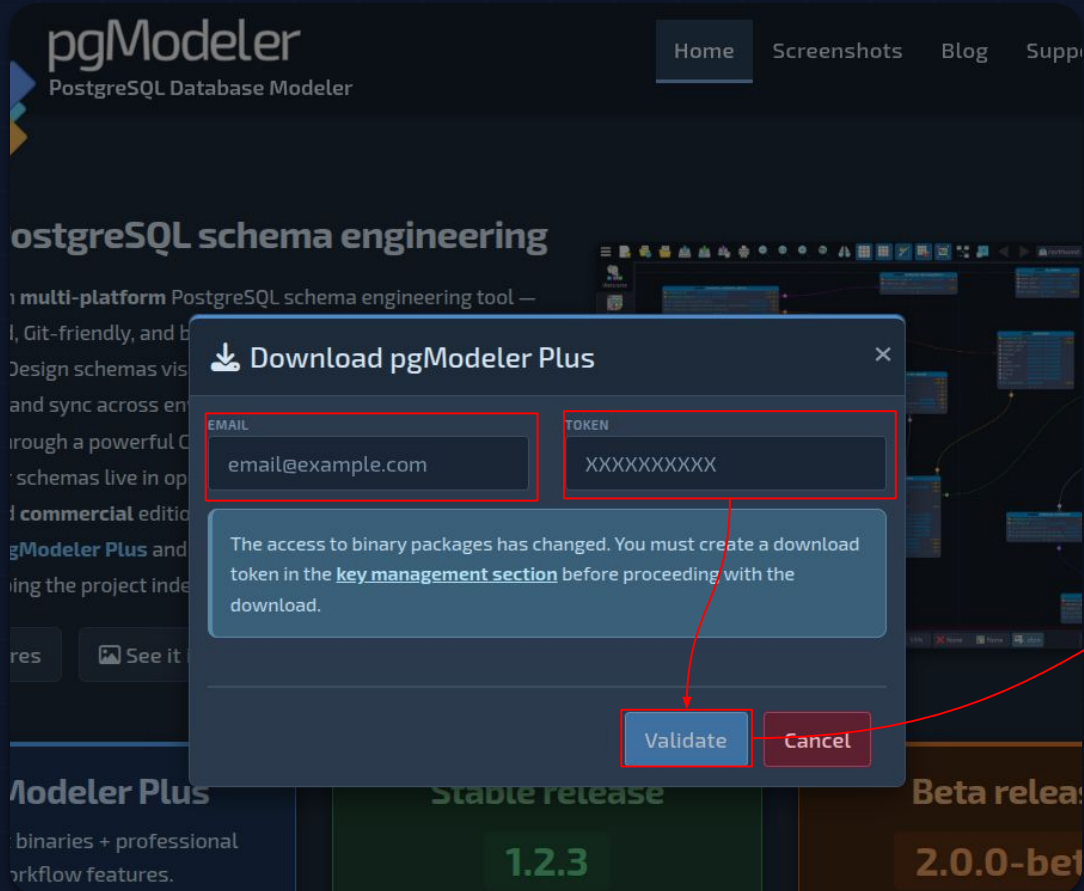
Encerramento do workshop com um momento para perguntas e respostas sobre o conteúdo abordado.



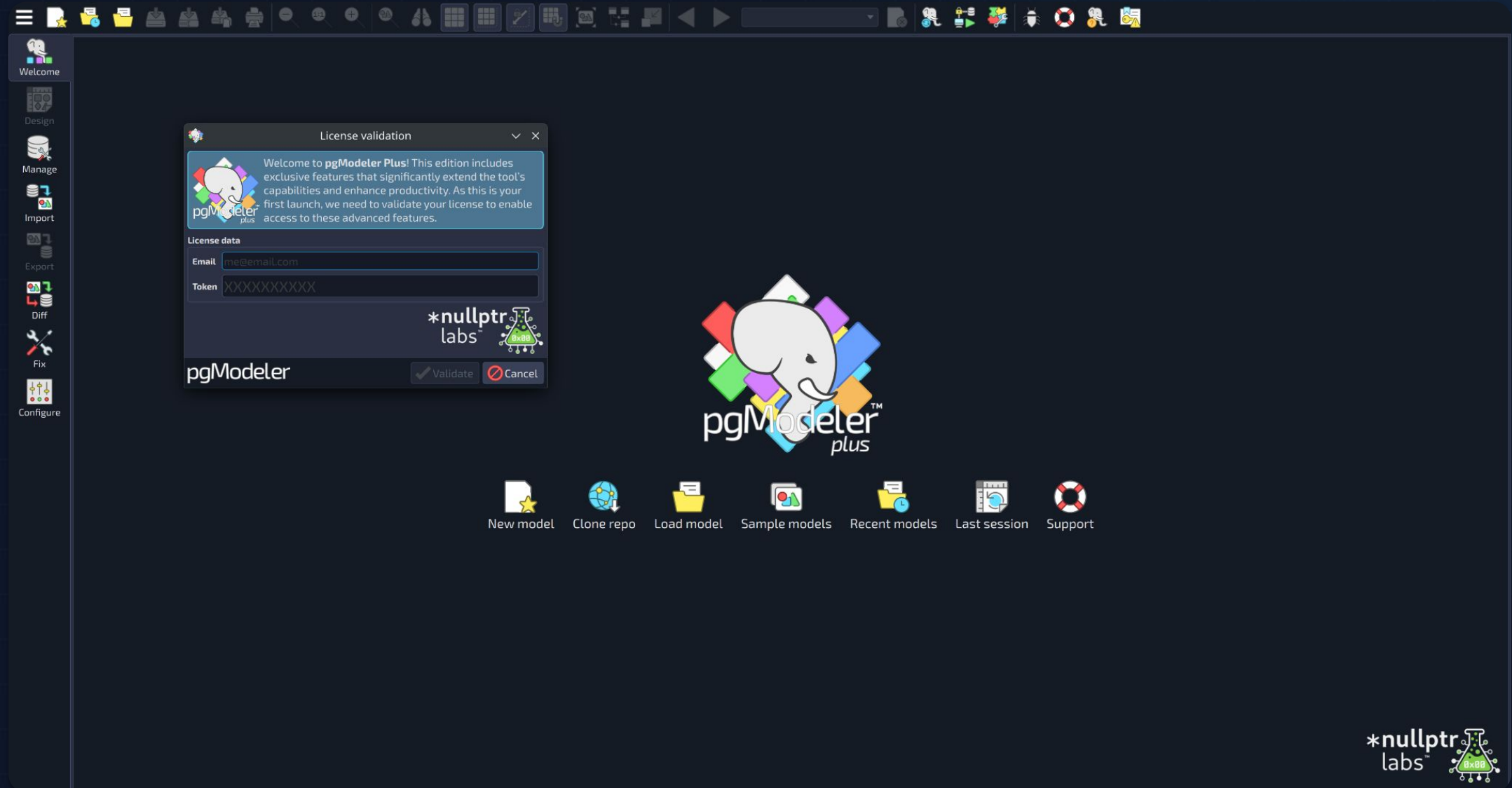
Instalação e tour pela ferramenta

Instalação e tour pela ferramenta

 <https://pgmodeler.io/download>



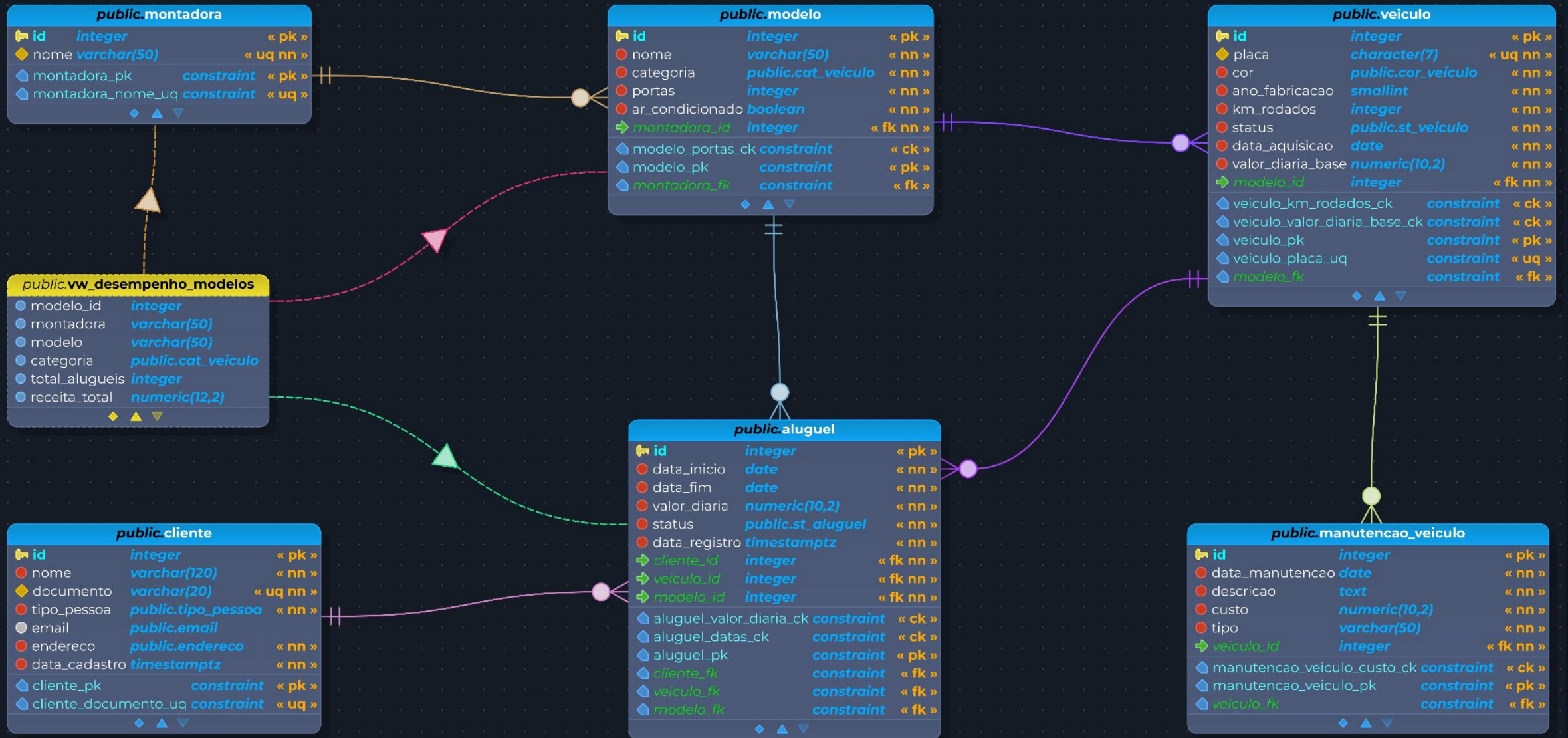
Instalação e tour pela ferramenta





Modelagem do banco de dados “rentacar”

Banco de dados "rentacar": Visão geral



Banco de dados “rentacar”: Tabelas

public.montadora			
 id	integer	« pk »	
 nome	varchar(50)	« nn »	
 montadora_pk	constraint	« pk »	

public.modelo			
 id	integer	« pk »	
 nome	varchar(50)	« nn »	
 portas	integer	« nn »	
 ar_condicionado	boolean	« nn »	
 modelo_pk	constraint	« pk »	

public.veiculo			
 id	integer	« pk »	
 placa	character(7)	« nn »	
 ano_fabricacao	smallint	« nn »	
 km_rodados	integer	« nn »	
 data_aquisicao	date	« nn »	
 valor_diaria_base	numeric(10,2)	« nn »	
 veiculo_pk	constraint	« pk »	

Colunas “id”:
Marcar como
IDENTITY ALWAYS

Default value

☐ Expression ☐ Generated

☐ Sequence ☐

☒ Identity ALWAYS ☐ NOT NULL

public.cliente			
 id	integer	« pk »	
 nome	varchar(120)	« nn »	
 documento	varchar(20)	« nn »	
 data_cadastro	timestampz	« nn »	
 cliente_pk	constraint	« pk »	

public.aluguel			
 id	integer	« pk »	
 data_inicio	date	« nn »	
 data_fim	date	« nn »	
 valor_diaria	numeric(10,2)	« nn »	
 data_registro	timestampz	« nn »	
 aluguel_pk	constraint	« pk »	

public.manutencao_veiculo			
 id	integer	« pk »	
 data_manutencao	date	« nn »	
 descricao	text	« nn »	
 custo	numeric(10,2)	« nn »	
 tipo	varchar(50)	« nn »	
 manutencao_veiculo_pk	constraint	« pk »	

Banco de dados “rentacar”: Types e domain

public.montadora		
🚗 id	integer	« pk »
🔴 nome	varchar(50)	« nn »
🔵 montadora_pk	constraint	« pk »

public.modelo		
🚗 id	integer	« pk »
🔴 nome	varchar(50)	« nn »
🔴 categoria	public.cat_veiculo	« nn »
🔴 portas	integer	« nn »
🔴 ar_condicionado	boolean	« nn »
🔵 modelo_pk	constraint	« pk »

public.veiculo		
🚗 id	integer	« pk »
🔴 placa	character(7)	« nn »
🔴 cor	public.cor_veiculo	« nn »
🔴 ano_fabricacao	smallint	« nn »
🔴 km_rodados	integer	« nn »
🔴 status	public.st_veiculo	« nn »
🔴 data_aquisicao	date	« nn »
🔴 valor_diaria_base	numeric(10,2)	« nn »
🔵 veiculo_pk	constraint	« pk »

ENUM TYPE:
hatch, sedan, suv,
picape, minivan,
esportivo, van

ENUM TYPE:
prata,
branco,
preto,
cinza,
vermelho,
azul

ENUM TYPE:
disponivel,
alugado,
manutencao,
inativo

DOMAIN:
VALUE = 'F' OR
VALUE = 'J'

public.cliente		
🚗 id	integer	« pk »
🔴 nome	varchar(120)	« nn »
🔴 documento	varchar(20)	« nn »
🔴 tipo_pessoa	public.tipo_pessoa	« nn »
🔴 email	public.email	« nn »
🔴 endereco	public.endereco	« nn »
🔴 data_cadastro	timestampz	« nn »
🔵 cliente_pk	constraint	« pk »

public.aluguel		
🚗 id	integer	« pk »
🔴 data_inicio	date	« nn »
🔴 data_fim	date	« nn »
🔴 valor_diaria	numeric(10,2)	« nn »
🔴 status	public.st_aluguel	« nn »
🔴 data_registro	timestampz	« nn »
🔵 aluguel_pk	constraint	« pk »

ENUM TYPE:
reservado,
em_andamento,
concluido,
cancelado

public.manutencao_veiculo		
🚗 id	integer	« pk »
🔴 data_manutencao	date	« nn »
🔴 descricao	text	« nn »
🔴 custo	numeric(10,2)	« nn »
🔴 tipo	varchar(50)	« nn »
🔵 manutencao_veiculo_pk	constraint	« pk »

Banco de dados "rentacar": Constraints

public.montadora		
id	integer	« pk »
nome	varchar(50)	« uq nn »
montadora_pk	constraint	« pk »
montadora_nome_uq	constraint	« uq »

UNIQUE:
nome

public.modelo		
id	integer	« pk »
nome	varchar(50)	« nn »
categoria	public.cat_veiculo	« nn »
portas	integer	« nn »
ar_condicionado	boolean	« nn »
modelo_portas_ck	constraint	« ck »
modelo_pk	constraint	« pk »

CHECK:
portas >= 2

CHECK:
km_rodados >= 0

public.veiculo		
id	integer	« pk »
placa	character(7)	« uq nn »
cor	public.cor_veiculo	« nn »
ano_fabricacao	smallint	« nn »
km_rodados	integer	« nn »
status	public.st_veiculo	« nn »
data_aquisicao	date	« nn »
valor_diaria_base	numeric(10,2)	« nn »
veiculo_km_rodados_ck	constraint	« ck »
veiculo_valor_diaria_base_ck	constraint	« ck »
veiculo_pk	constraint	« pk »
veiculo_placa_uq	constraint	« uq »

CHECK:
valor_diaria_base > 0

UNIQUE:
placa

public.cliente		
id	integer	« pk »
nome	varchar(120)	« nn »
documento	varchar(20)	« uq nn »
tipo_pessoa	public.tipo_pessoa	« nn »
email	public.email	« nn »
endereco	public.endereco	« nn »
data_cadastro	timestampz	« nn »
cliente_pk	constraint	« pk »
cliente_documento_uq	constraint	« uq »

UNIQUE:
documento

CHECK:
valor_diaria > 0

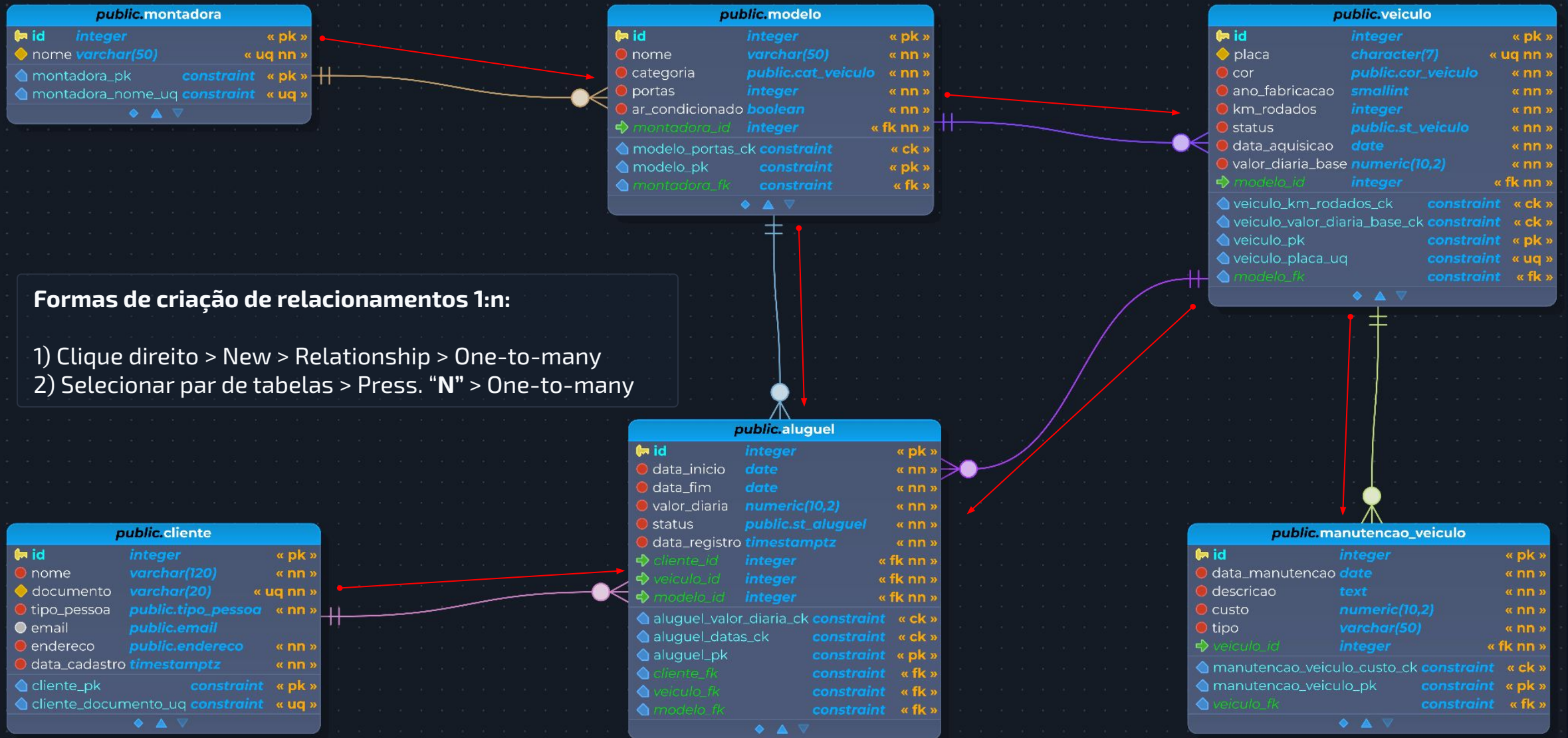
public.aluguel		
id	integer	« pk »
data_inicio	date	« nn »
data_fim	date	« nn »
valor_diaria	numeric(10,2)	« nn »
status	public.st_aluguel	« nn »
data_registro	timestampz	« nn »
aluguel_valor_diaria_ck	constraint	« ck »
aluguel_datas_ck	constraint	« ck »
aluguel_pk	constraint	« pk »

CHECK:
data_fim >= data_inicio

CHECK:
custo >= 0

public.manutencao_veiculo		
id	integer	« pk »
data_manutencao	date	« nn »
descricao	text	« nn »
custo	numeric(10,2)	« nn »
tipo	varchar(50)	« nn »
manutencao_veiculo_custo_ck	constraint	« ck »
manutencao_veiculo_pk	constraint	« pk »

Banco de dados "rentacar": Relacionamentos



Formas de criação de relacionamentos 1:n:

- 1) Clique direito > New > Relationship > One-to-many
- 2) Selecionar par de tabelas > Press. "N" > One-to-many

Banco de dados “rentacar”: View

public.vw_desempenho_modelos

● modelo_id	<i>integer</i>
● montadora	<i>varchar(50)</i>
● modelo	<i>varchar(50)</i>
● categoria	<i>public.cat_veiculo</i>
● total_alugueis	<i>integer</i>
● receita_total	<i>numeric(12,2)</i>



```
1 SELECT
2   @{tb_mod}.{m_id} AS @{m_id},
3   @{tb_mon}.{mon_nome} AS @{mon_nome},
4   @{tb_mod}.{m_nome} AS @{m_nome},
5   @{tb_mod}.{m_categoria},
6   COUNT(@{tb_al}.{al_id}) AS total_alugueis,
7   COALESCE(
8     SUM(
9       CASE
10        WHEN @{tb_al}.{al_status} IN ('concluido', 'em_andamento')
11        THEN ((@{tb_al}.{al_dt_fim} -
12              @{tb_al}.{al_dt_inicio} + 1) *
13              @{tb_al}.{al_val_diaria})
14        ELSE 0
15       END
16     ), 0.00
17   )::numeric(12,2) AS receita_total
18 FROM {tb_mod} AS @{tb_mod}
19 JOIN {tb_mon} AS @{tb_mon}
20   ON @{tb_mod}.{m_mon_id} = @{tb_mon}.{mon_id}
21 LEFT JOIN {tb_al} AS @{tb_al}
22   ON @{tb_al}.{al_mod_id} = @{tb_mod}.{m_id}
23 GROUP BY @{tb_mod}.{m_id}, @{tb_mon}.{mon_nome},
24          @{tb_mod}.{m_nome}, @{tb_mod}.{m_categoria}
25 ORDER BY total_alugueis DESC, receita_total DESC
26
```

Code References SQL preview

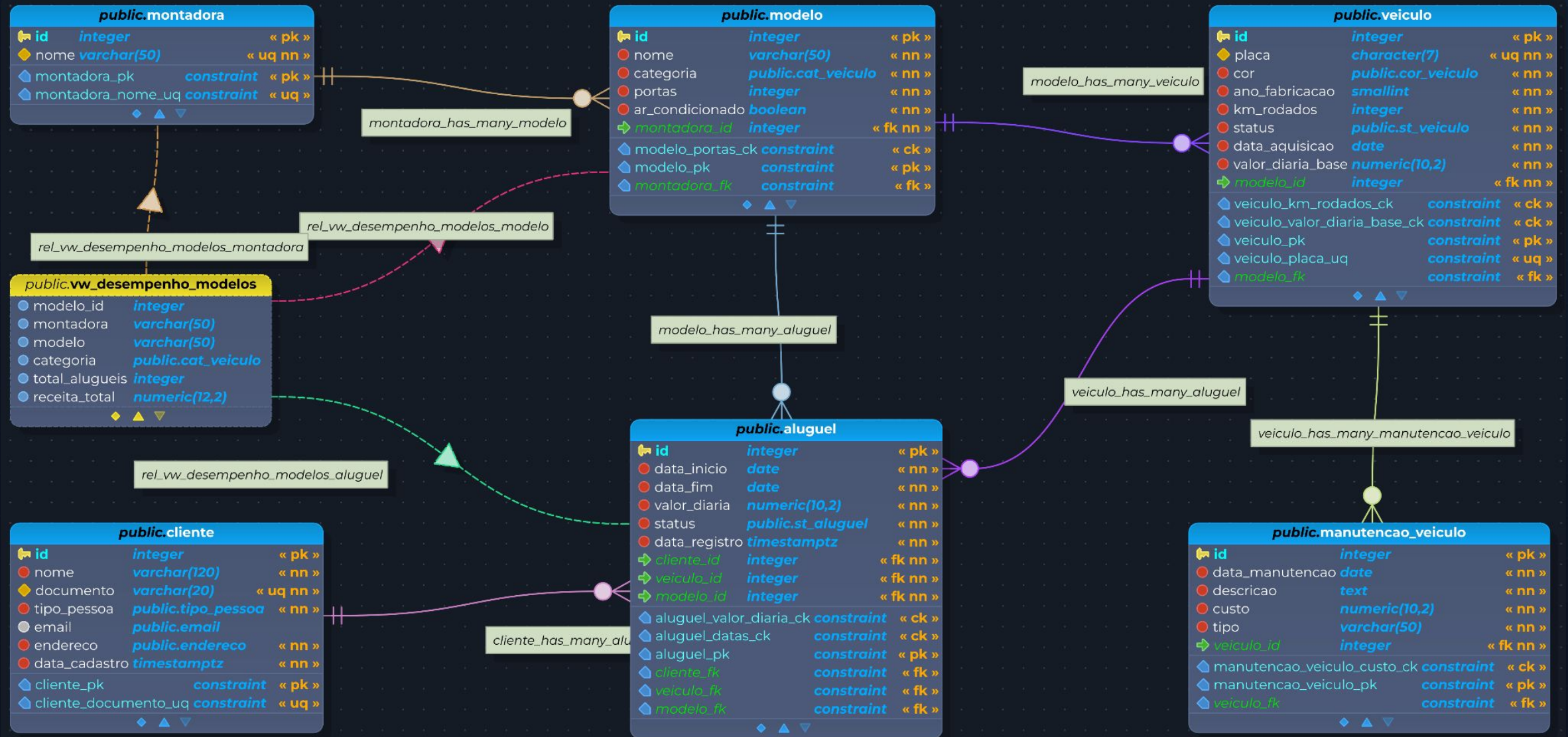
Banco de dados “rentacar”: View

public.vw_desempenho_modelos

- modelo_id integer
- montadora varchar(50)
- modelo varchar(50)
- categoria public.cat_veiculo
- total_alugueis integer
- receita_total numeric(12,2)

Ref. name	Ref. alias	Object	Type	Use signature	Format name	Use column(
tb_mon	mon	public.montadora	Table	Yes	Yes	No
tb_mod	m	public.modelo	Table	Yes	Yes	No
tb_al	a	public.aluguel	Table	Yes	Yes	No
al_id		public.aluguel.id	Column	No	Yes	No
al_dt_inicio		public.aluguel.data_inicio	Column	No	No	No
al_dt_fim		public.aluguel.data_fim	Column	No	No	No
al_status		public.aluguel.status	Column	No	No	No
al_val_diaria		public.aluguel.valor_diaria	Column	No	No	No
al_mod_id		public.aluguel.modelo_id	Column	No	No	No
m_id	modelo_id	public.modelo.id	Column	No	No	Yes
m_mon_id		public.modelo.montadora_id	Column	No	No	No
mon_nome	montadora	public.montadora.nome	Column	No	No	Yes
m_nome	modelo	public.modelo.nome	Column	No	No	Yes
m_categoria		public.modelo.categoria	Column	No	No	Yes
mon_id		public.montadora.id	Column	No	No	No

Banco de dados "rentacar": Resultado final



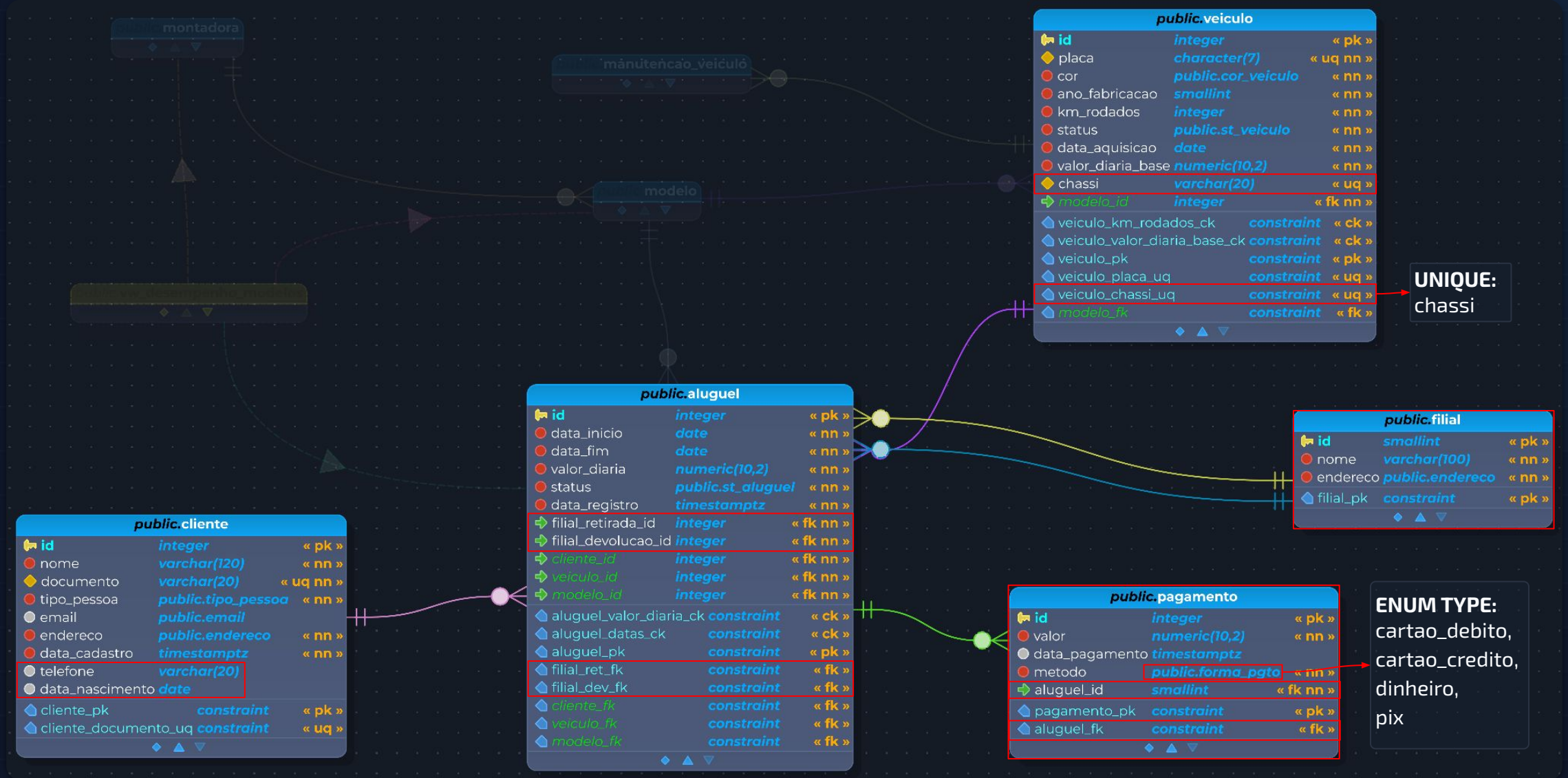


Exportação e engenharia reversa



Comparação de banco de dados (diff)

Banco de dados “rentacar” modificado



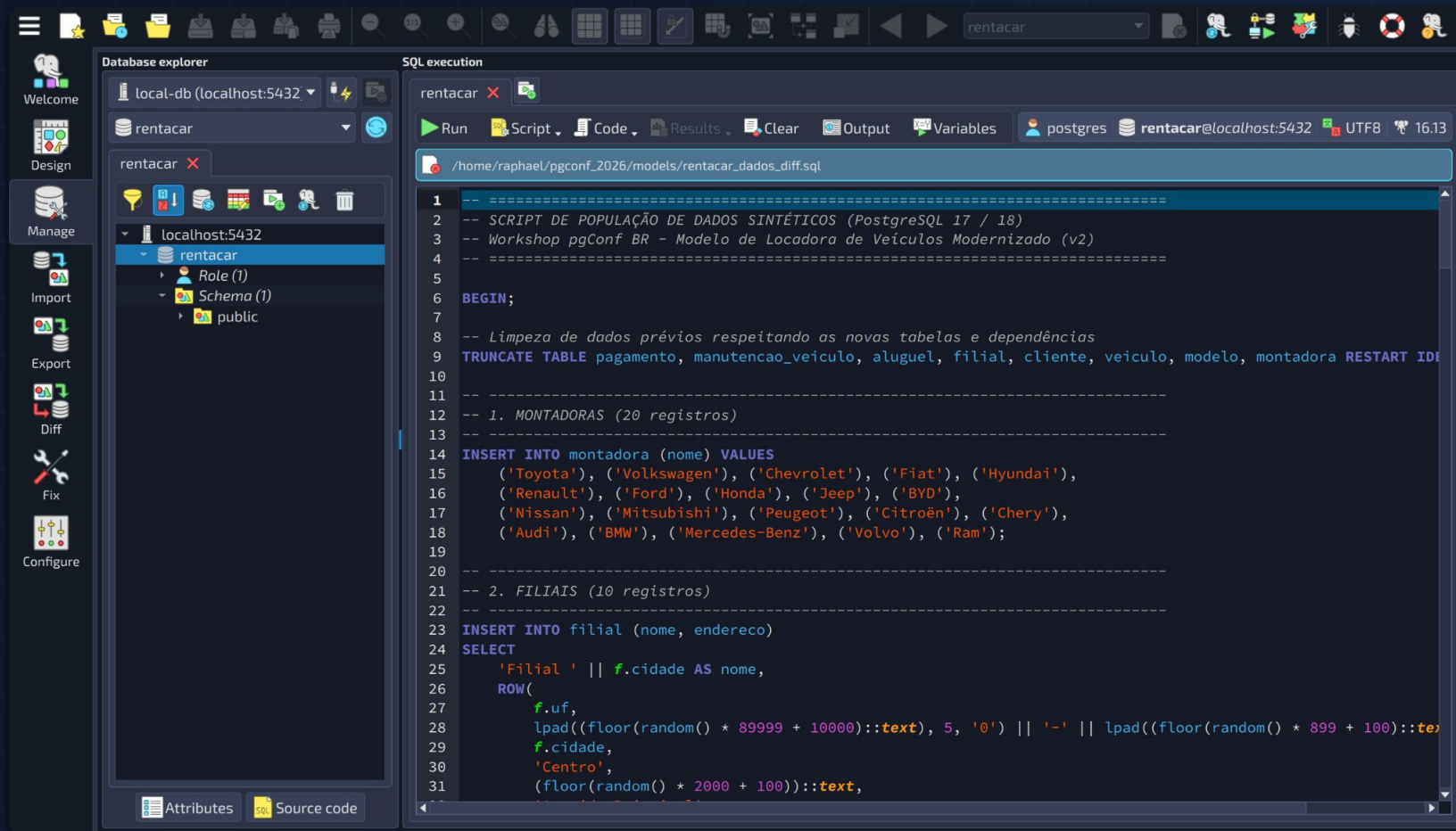


Administração de banco de dados

Populando o banco de dados "rentacar"

 <https://github.com/rkhaotix/pgconfbr26>

1. Baixar arquivo **rentacar_dados_diff.sql**
2. Configurar conexão com o banco de dados
3. Selecionar seu banco de dados **rentacar**
4. Abrir nova aba de execução de comandos SQL (Ctrl+T)
5. Clicar em **Script > Load** e selecionar o arquivo
6. Executar via comando **Run** (F5)





Ferramenta de linha de comando

Ferramenta de linha de comando

Exportação para script SQL

```
$> pgmodeler-cli -ef -if rentacar.dbm -of  
rentacar.sql
```

Exportação para dicionário de dados

```
$> pgmodeler-cli -ec -if rentacar.dbm -of  
rentacar.html
```

Exportação para imagem PNG

```
$> pgmodeler-cli -ep -if rentacar.dbm -of  
rentacar.png
```

Exportação para arquivo SVG

```
$> pgmodeler-cli -es -if rentacar.dbm -of  
rentacar.svg
```

Exportação para o banco de dados

```
$> pgmodeler-cli --list-conns
```

```
$> pgmodeler-cli -ed -if rentacar.dbm -ca  
local-db -dd -ff
```

```
$> pgmodeler-cli -ed -if rentacar.dbm -H  
localhost -p 5432 -u postgres -w postgres -D  
postgres -dd -ff
```

Ferramenta de linha de comando

Engenharia reversa de banco de dados

```
$> pgmodeler-cli --list-conns

$> pgmodeler-cli -im -id rentacar -ca
local-db -of rentacar_imported.dbm

$> pgmodeler-cli -im -id rentacar -of
rentacar_imported -H localhost -p 5432 -u
postgres -w postgres -D postgres
```

Comparação entre modelo e banco de dados

```
$> pgmodeler-cli --list-conns

$> pgmodeler-cli -df -if rentacar_diff.dbm
-cd rentacar -ca local-db --save -of
rentacar_diff.sql

$> pgmodeler-cli -df -if rentacar_diff.dbm
-cd rentacar -ca local-db --apply
```



Visão geral de plugins



Muito obrigado!

<https://pgmodeler.io>

<https://nullptrlabs.io>

raphael@pgmodeler.io